



实验（五）

综合程序设计

1、上机题目：将从键盘输入的N个无符号数保存到数组DAT，找出N个无符号数中的偶数存放数组P，统计并在终端上显示数组P的数据个数no。打印输出p和no

DAT word n dup (?)

P word n dup (?)

no word ?

- 要求：
- 1、求偶数数组P的功能用子程序实现
 - 2、画主程序及子程序流程图
 - 3、熟练掌握综合程序设计方法



实验 五

综合程序设计

2、上机题目：将从键盘输入的N个有符号数保存到数组TAB，找出N个有符号数中绝对值大于X的最小负奇数存放到Min单元，如果没有找到则Min单元存放0。在终端上显示Min的绝对值。

TAB Dword n dup(?)

X Dword xx(无符号数，自己设定)

Min Dword ?

- 要求：
- 1、求数据Min的功能用子程序实现
 - 2、画主程序及子程序流程图
 - 3、熟练掌握综合程序设计方法



提示

- ◆ 判断偶数的方法：使用 AND 指令对目标操作数与 01h（或 1）进行按位与运算。若运算结果为 0，则说明该数为偶数；否则为奇数。



汇编语言程序设计

LEA指令

地址传送指令LEA (Load Effective Address)

- ◆ ➤ 地址传送指令获取存储器操作数的地址
- ◆ **LEA r16/r32, mem**
- ◆ ;r16/r32 ← mem的有效地址EA（不需类型一致）
- ◆ ;IA-32使用32位地址，保存于32位通用寄存器r32





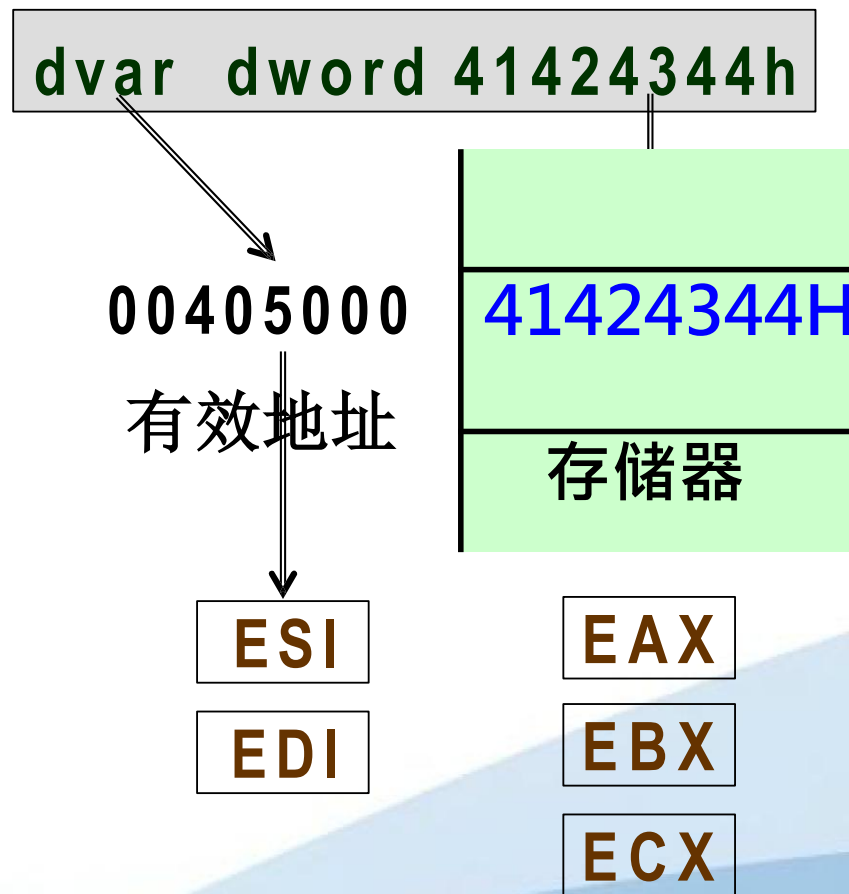
LEA指令类似地址操作符 OFFSET的作用

- LEA指令在指令执行时计算出偏移地址
 - ▶ OFFSET操作符在汇编阶段取得变量的偏移地址
- OFFSET无需在执行时计算、指令执行速度更快
 - ▶ LEA指令能获取汇编时变量的偏移地址

```
lea esi, var  
mov edi, offset var
```

地址传送程序

```
mov eax,dvar  
lea esi,dvar  
mov ebx,[esi]  
mov edi,offset dvar  
mov ecx,[edi]
```





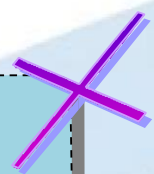
用LEA指令实现运算功能

```
lea edx,[esi+edi*4+100h]
```

```
;EDX=ESI+EDI×4+100H
```

- LEA指令在计算地址时，可进行加和移位操作
 - ▶ 编译器将其用于实现加法，或者乘以2、4和8
- 不能用OFFSET操作符实现

```
lea edx,[esi+edi*4+100h]
```



LEA指令总结

- ◆ ➤ LEA指令获得存储器操作数的有效地址
 - ◆ ➤ 在LEA指令执行时计算地址
 - ◆ ➤ 对任何存储器寻址方式都可用
- ◆ ➤ 对存储器的直接寻址
 - ◆ ➤ 建议用OFFSET操作符在汇编阶段获得地址
- ◆ ➤ 对存储器的其他寻址
 - ◆ ➤ 只能使用LEA指令获得地址

求补指令NEG (negative)

- 对操作数执行求补运算，即用零减去操作数
NEG reg/mem ; $\text{reg/mem} \leftarrow 0 - \text{reg/mem}$
- 对标志的影响与用零作减法的SUB指令一样
- 可用于对负数求补码或由补码求其绝对值

0-i

```
neg al  
neg byte ptr [ebx]  
neg wvar[esi]
```

```
neg ax  
neg word ptr [ebx]  
neg wvar[edi]
```

Pushad和popad

- ◆ PUSHAD和POPAD是一对互逆的操作指令，通常成对出现。
- ◆ PUSHAD指令在堆栈中按顺序压入下列寄存器:EAX,ECX,EDX,EBX,ESP,EBP,ESI和EDI.
- ◆ POPAD指令则是PUSHAD指令的逆操作。POPAD指令按照与上面相反的顺序依次弹出寄存器的值。顺序为EDI,ESI,EBP,ESP,EBX,EDX,ECX,EAX.



堆栈传参和寄存器传参

- ◆ 慎用 pushad和popad;
- ◆ 堆栈传参, ebp和esp都被使用, 存在重复入栈保护;
- ◆ 寄存器传参, 如果统一出栈会把子程序计算的结果恢复覆盖而无法传回。
- ◆ 建议除非非常必要, 否则不建议直接采用这两条指令来入栈和出栈保护。

1.3.3 X86源程序框架

;eg0000.asm in Windows Console

include irvine32.inc

;包含32位输入输出文件

常量1 equ xxxx;

;根据需要定义常量，**可以没有**

常量2 equ xxxx;

.data

;定义数据段

org XX

; 指定变量的存储偏移量，可以没有

.....

;数据定义（数据待填）

.code

;定义代码段

main proc

;主过程开始

.....

;主程序（指令待填）

exit

;程序正常执行结束

main endp

;主过程结束

.....

;子过程（指令待填）

end main

;汇编结束

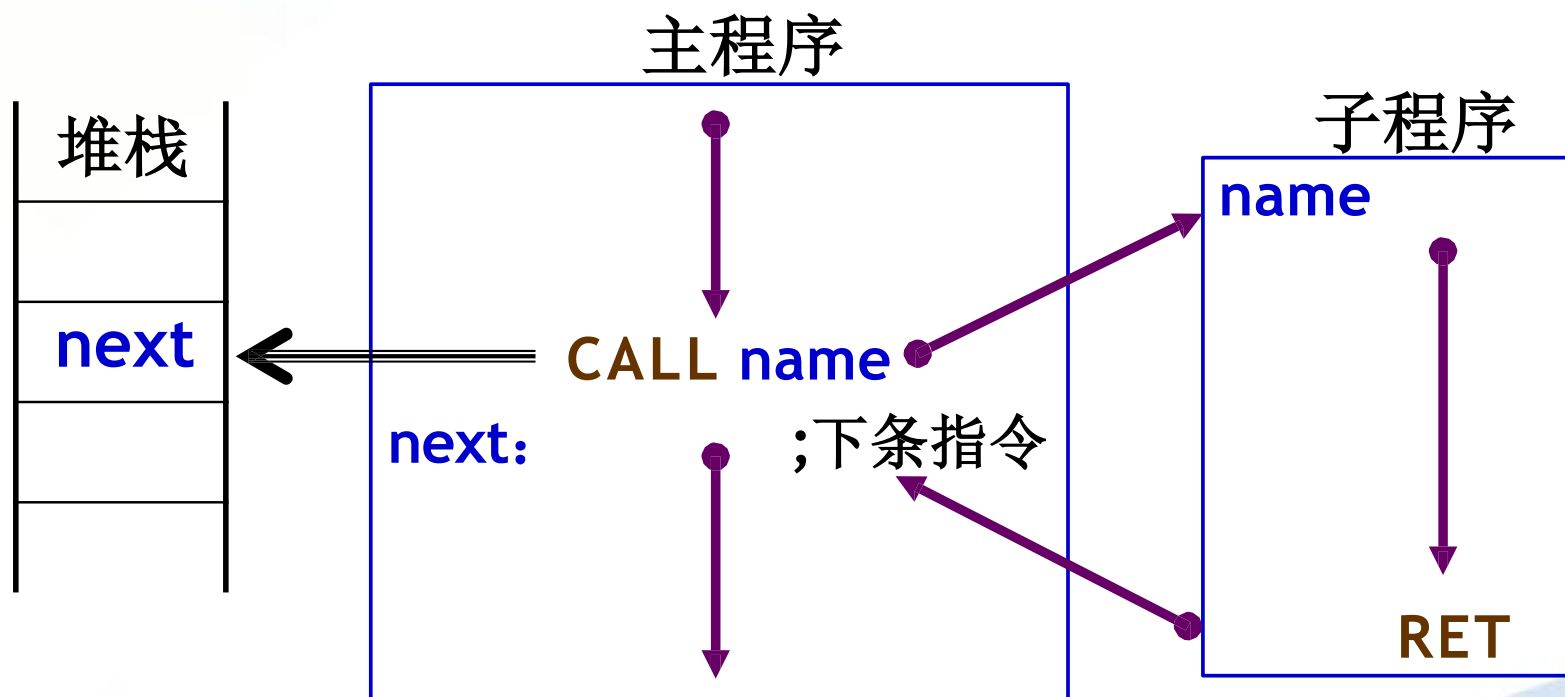
;通常从主过程的第一条指令开始执行





汇编语言程序设计

子程序设计





过程定义伪指令

- MASM利用过程定义伪指令获得子程序信息

过程名 **PROC**

;过程体

过程名 **ENDP**

;过程名为符合语法的标识符

MASM会根据存储模型等信息确定子程序的远近调用，并相应产生调用、返回指令

子程序框架

标识符

proc

;过程定义（子程序开始）

push ...1

;保护寄存器

push ...2

;子程序体

pop ...2

;恢复寄存器

pop ...1

;子程序返回

ret

标识符

endp

;过程（子程序）结束

寄存器传递参数

;代码段，主程序

again: **rol** **eax**,4
push **eax**
call **htonasc**
mov **regd+4[ebx]**,**al**
pop **eax**
inc **ebx**
loop **again**

AL

AL

;代码段，子程序

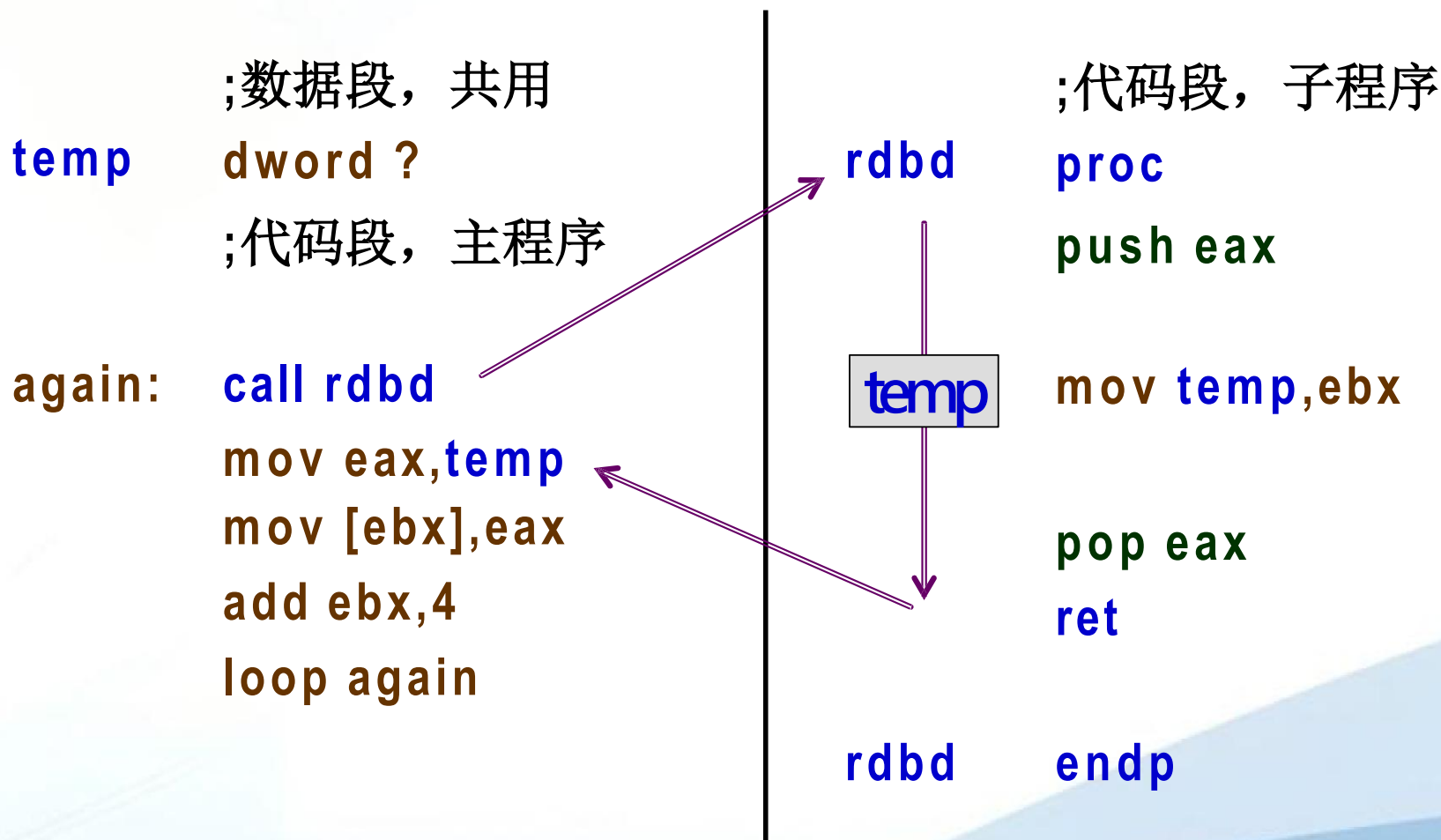
htonasc **proc**

and **al**,0fh
or **al**,30h
cmp **al**,39h
jbe **htonend**
add **al**,7

htonend: **ret**

htonasc **endp**

共享变量传递参数





子程序框架（堆栈传递参数）

标识符

<code>proc</code>	;过程定义（子程序开始）
<code>push ebp</code>	; 堆栈传递入口参数
<code>mov</code>	
<code>ebp,esp</code>	;保护寄存器
<code>push ...1</code>	
<code>push ...2</code>	
	;子程序体
<code>pop ...2</code>	;恢复寄存器
<code>pop ...1</code>	
<code>pop ebp</code>	
<code>ret</code>	;子程序返回
<code>endp</code>	;过程（子程序）结束

标识符



堆栈传递参数

;代码段, 主程序

push lengthof array; 元素个数
push offset array; array 首地址
call mean

call dispsid

主程序平衡堆栈

;代码段,

mean proc

push ebp

mov ebp, esp

...

mov ecx, [ebp+12] ; 个数

mov ebx, [ebp+8]; 首地址

...

...

mean

pop ebp

ret 8

endp



Jcc跳转指令汇总(16条)

条件分类	标志位	标志位	指令	助记单词	指令全称
单标志位	ZF	ZF=1 ZF=0	JZ JNZ	Zero	Jump if Zero Jump if Not Zero
	CF	CF=1 CF=0	JC JNC	Carry Flag	Jump if Carry Jump if Not Carry
	SF	SF=1 SF=0	JS JNS	Sign	Jump if Sign Jump if Not Sign
	OF	OF=0 OF=1	JO JNO	Overflow	Jump if Overflow Jump if Not Overflow
	PF	PF=1 PF=0	JP JPE JNP JPO	Parity	Jump if Parity Jump if Parity Even Jump if Not Parity Jump if Parity Odd
两数大小关系	无符号数: 低于、不高于等于 不低于、高于等于 低于等于、不高于 不低于等于、高于	CF=1 CF=0 CF=1或ZF=1 CF=0且ZF=0	JB/JNAE JNB/JAE JBE/JNA JNBE/JA	Above Below Equal	Jump if Below / jump if Not Above or Equal Jump if Not Below / Jump if Above or Equal Jump if Below or Equal / Jump if Not Above Jump if Not Below or Equal / Jump if Above
	有符号数: 小于、不大于等于 不小于、大于等于 小于等于、不大于 不小于等于、大于	SF≠OF SF=OF SF≠0或ZF=1 SF=0且ZF=0	JL/JNGE JNL/JGE JLE/JNG JNLE/JG	Greater Less	Jump if Less / Jump if Not Greater or Equal Jump if Not Less / Jump if Greater or Equal Jump if Less or Equal / Jump if Not Greater Jump if Not Less or Equal / Jump if Greater

产生条件的指令

➤常用指令1：比较指令CMP

- ▶ 进行减法运算

- ▶ 用于判断两个数据大小、是否相等

➤常用指令2：测试指令TEST

- ▶ 进行逻辑与运算

- ▶ 用于判断某位为0或为1等

➤其他指令：能够影响状态标志的指令 加减运算指令、逻辑运算指令、移位指令等



字符串复制

;数据段

srcmsg byte 'Try your best, why not.',0

dstmsg byte sizeof srcmsg dup(?)

srcmsg

'T'	'r'	'y'		'n'	'o'	't'	'.'	0
-----	-----	-----	--	-----	-----	-----	-----	---

dstmsg

--	--	--	--	--	--	--	--	--

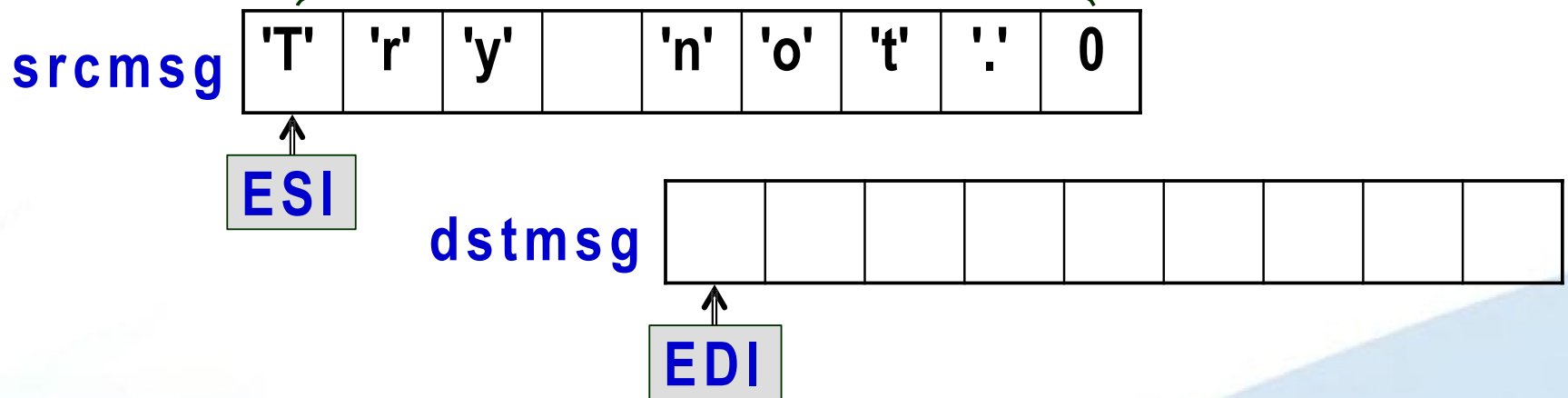
寄存器指向字符

```
mov ecx,lengthof srcmsg
```

```
mov esi,offset srcmsg
```

```
mov edi,offset dstmsg
```

ECX=字符个数



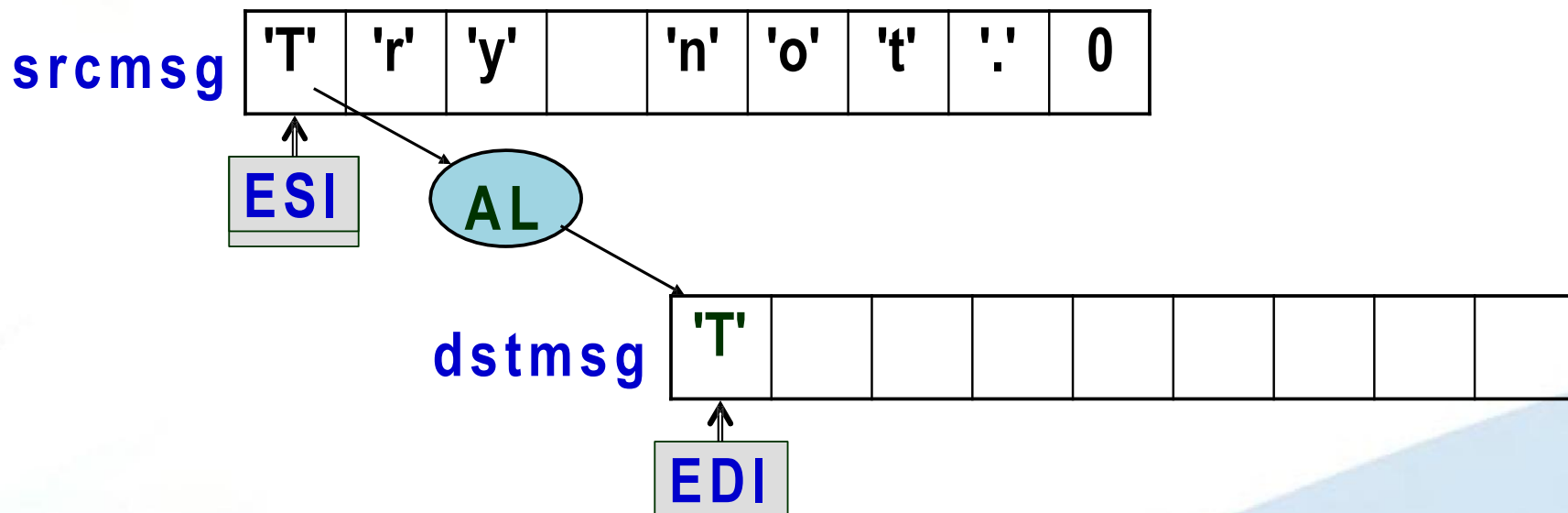
字符传送

mov al, [esi]

;取源串一个字符送AL

mov [edi], al

;将AL传送给目的串位置



指向下一个字符

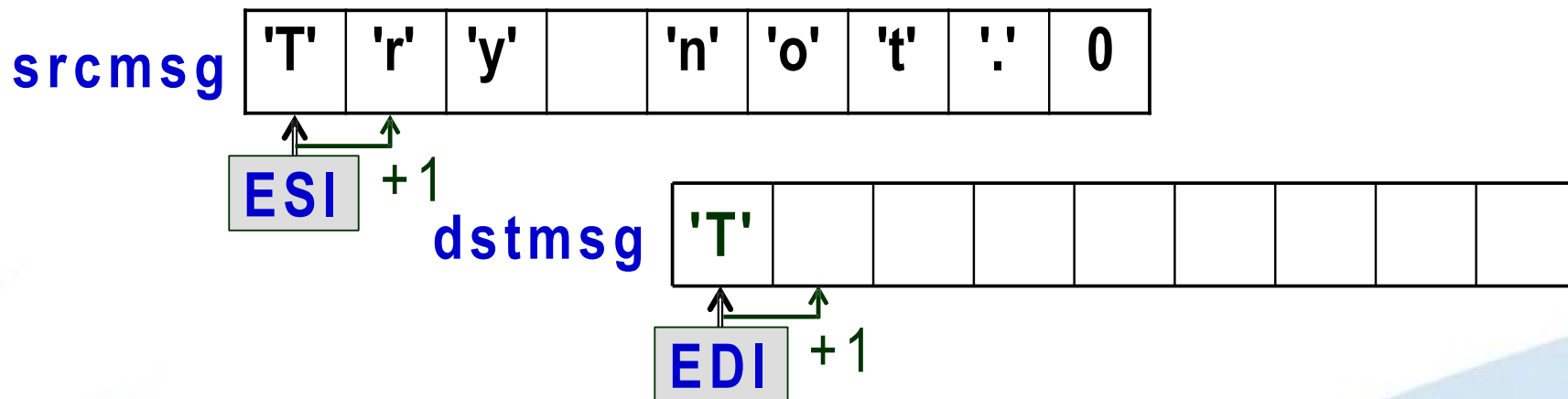
add esi,1

;源串指针加1

add edi,1

;目的串指针加1

ADD是加法指令





字符串复制：逐个字符传送

again: **mov al, [esi]**

;传送一个字符

;指向下一个字符

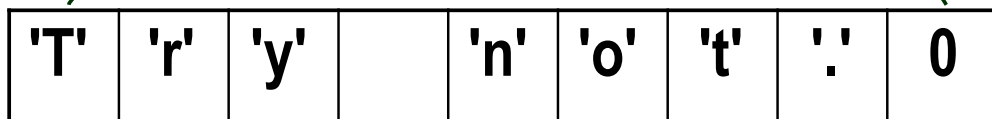
loop again

;字符个数减1，不为0继续

LOOP是循环指令

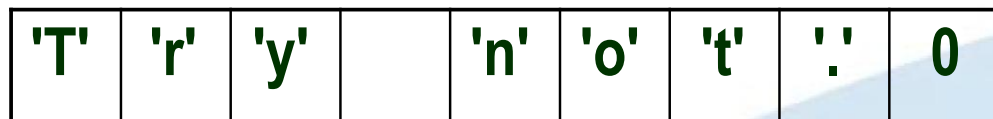
ECX = 字符个数

srcmsg



ESI

dstmsg



EDI



寄存器间接寻址程序

;数据段

```
srcmsg    byte 'Try your best, why not.',0
dstmsg    byte sizeof srcmsg dup(?)
```

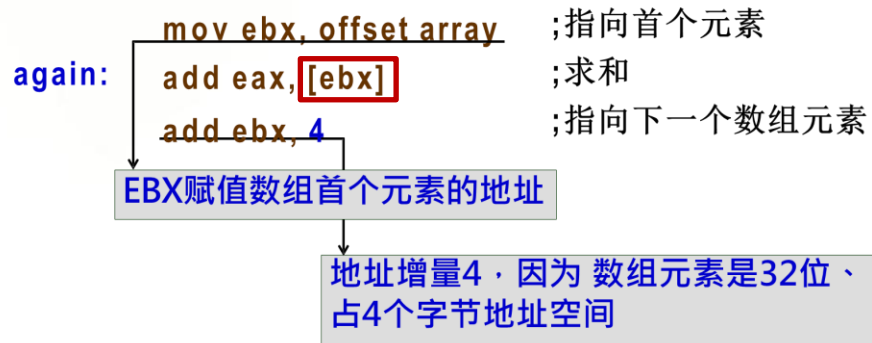
;代码段

```
mov ecx,lengthof srcmsg
mov esi,offset srcmsg
mov edi,offset dstmsg
again:    mov al, [esi]
          mov [edi], al
          add esi,1
          add edi,1
          loop again
          mov esi,offset dstmsg
          call writestring
```

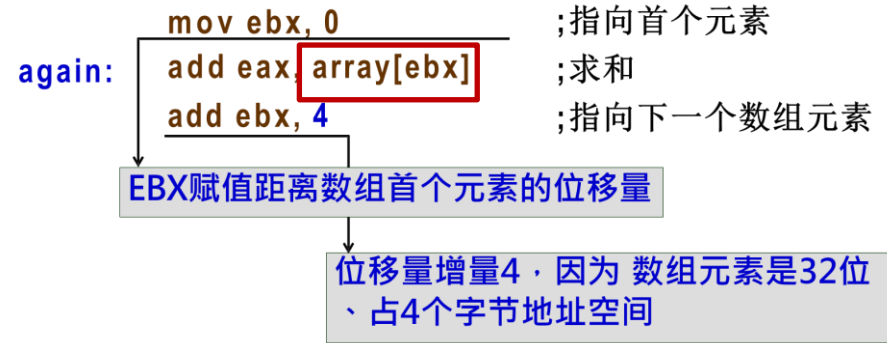


三种数组元素的访问方法

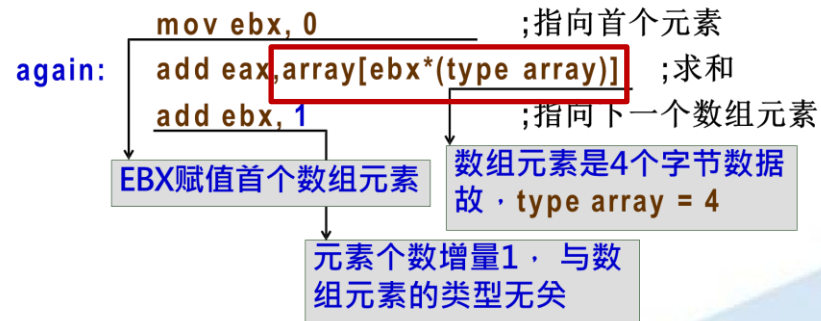
寄存器间接寻址访问数组元素



寄存器相对寻址访问数组元素



寄存器（带比例）变址寻址访问数组元素



输入输出函数

- ◆ 输入：有符号整数，ReadInt，获得的数据存在eax中
 - ◆ ; Receives: nothing
 - ◆ ; Returns: If CF=0, the integer is valid, and EAX = binary value.
 - ◆ ; If CF=1, the integer is invalid and EAX = 0.
- ◆ 输出：WriteInt，准备的打印的数据放入eax中
 - ◆ ; Receives: EAX = the integer



输入输出函数

◆ 输入：ReadString

- ◆ ; Receives: EDX points to the input buffer,
- ◆ ; ECX contains the maximum string length
- ◆ ; Returns: EAX = size of the input string.

```
mov  edx, offset Linputarea      ; save offset in edx
mov  ecx, LMAX_DIGITS
call ReadString
mov  ecx, eax                    ; save length in CX
```

◆ 输出：WriteString

- ◆ ; output. Input parameter: EDX points to the string.

```
mov  edx, offset waitmsgstr
call WriteString
```



输入输出函数

◆ DumpMem

- ◆ ; Receives: ESI = starting offset, ECX = number of units,
- ◆ ; EBX = unit size (1=byte, 2=word, or 4=doubleword)
- ◆ ; Returns: nothing
- ◆ 例子:
- ◆ `mov esi,offset dat` ;esi存储内存开始地址
- ◆ `mov ebx,type dat` ;ebx存储数据类型字量
- ◆ `mov ecx,lengthof dat` ;ecx存储内存单元数量
- ◆ `call dumpmem`

其余的输入输出函数，详细见**Irvine32.asm**



OlllyDBG 常用快捷键

F2: 设置断点，只要在光标定位的位置按F2键即可，再按一次F2键则会删除断点。（相当于 SoftICE 中的 F9）

F8: 单步步过。每按一次这个键执行一条反汇编窗口中的一条指令，遇到 **CALL** 等子程序不进入其代码。（相当于 SoftICE 中的 F10）

F7: 单步步入。功能同单步步过(F8)类似，区别是遇到 **CALL** 等子程序时会进入其中，进入后首先会停留在子程序的第一条指令上。（相当于 SoftICE 中的 F8）

F4: 运行到选定位置。作用就是直接运行到光标所在位置处暂停。（相当于 SoftICE 中的 F7）



OlllyDBG 常用快捷键

F9: 运行。按下这个键如果没有设置相应断点的话，被调试的程序将直接开始运行。（相当于 SoftICE 中的 F5）

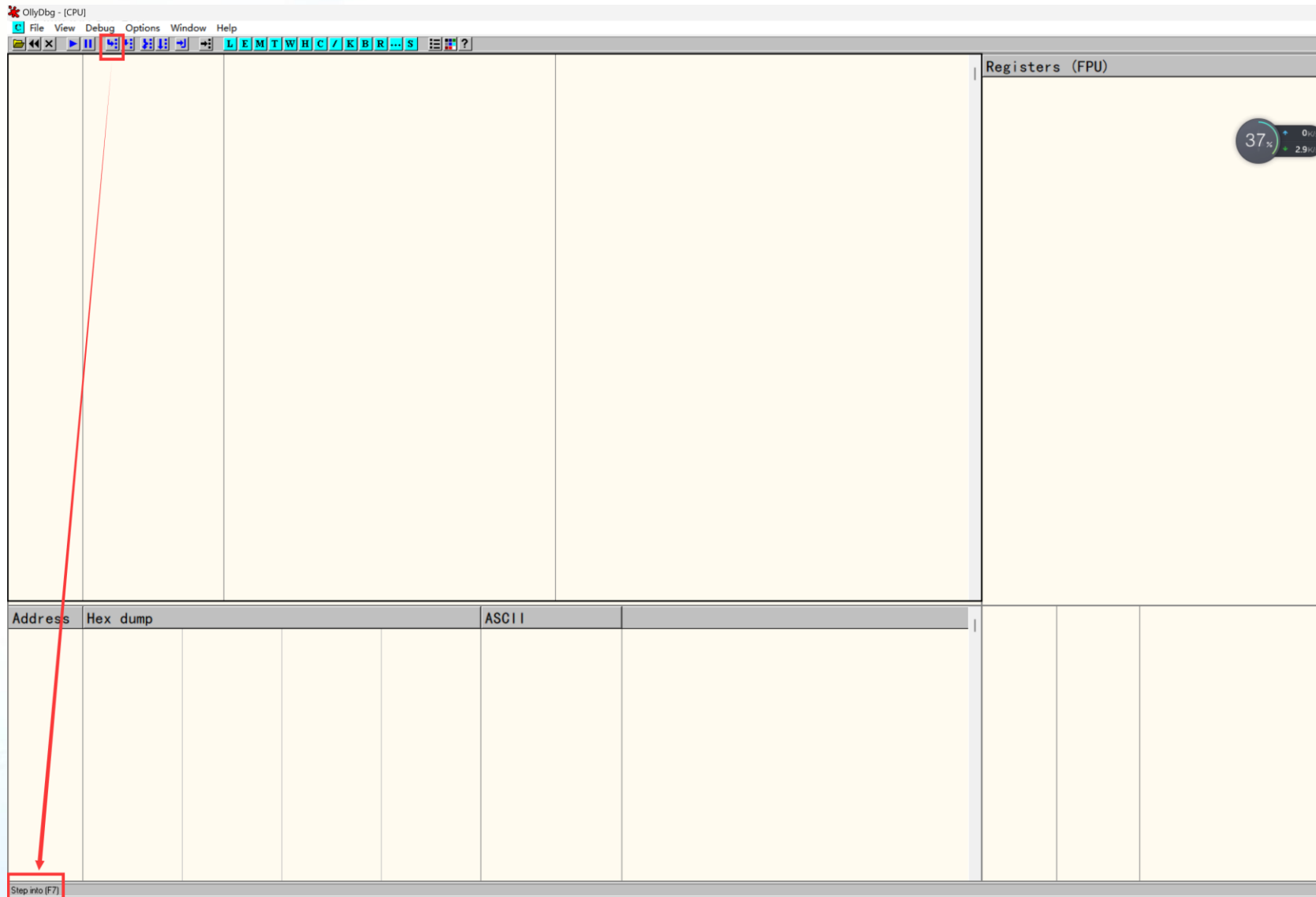
CTR+F9: 执行到返回。此命令在执行到一个 **ret** (返回指令)指令时暂停，常用于从系统领空返回到我们调试的程序领空。（相当于 SoftICE 中的 F12）

ALT+F9: 执行到用户代码。可用于从系统领空快速返回到我们调试的程序领空。（相当于 SoftICE 中的 F11）

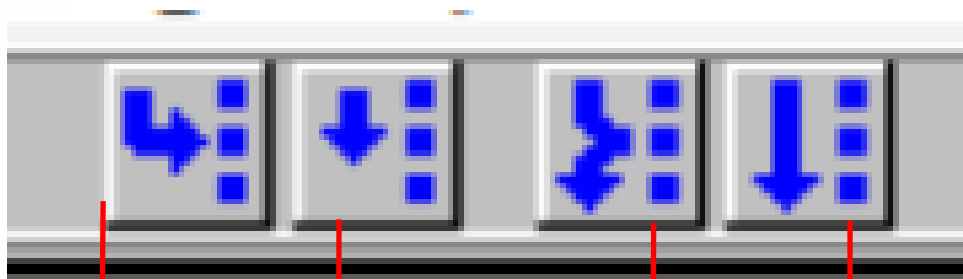
学习链接:

<https://www.cnblogs.com/xueyoo/articles/4938538.html>

调试的工具栏和快捷键



调试的工具栏和快捷键



F7: step into

F8: step over

Ctrl+F11:
trace into

Ctrl+F12:
trace over